

Evaluation modulaire

Cours collectifs

Brief client

Un petit studio de sport souhaite gérer simplement les inscriptions à ses cours collectifs.

Aujourd'hui, les réservations sont notées à la main. Le studio veut une première version très simple d'une application en ligne de commande pour suivre les cours proposés et le nombre de places disponibles.

L'objectif est de créer un outil minimal, mais fonctionnel, qui permet de gérer quelques cours dans une base de données locale.

Objectif de l'évaluation

Vous devez réaliser une petite application Python permettant de :

1. créer un cours
2. afficher les cours
3. inscrire une personne à un cours
4. afficher les cours complets
5. quitter l'application

L'application doit utiliser :

1. la programmation orientée objet
2. un ORM (SQLAlchemy)
3. une base SQLite locale (ou MySQL comme vous préférez - mais cela doit fonctionner avec SQLite)

Contexte fonctionnel

Chaque cours collectif contient les informations suivantes :

Champ	Description
<code>id</code>	identifiant unique du cours
<code>name</code>	nom du cours
<code>coach</code>	nom du coach
<code>capacity</code>	nombre maximum de participants
<code>registered_count</code>	nombre de participants inscrits
<code>price</code>	le tarif du cours

Fonctionnalités attendues

1. Créer un cours

L'utilisateur doit pouvoir ajouter un nouveau cours.

Le programme doit demander :

- Nom du cours
- Nom du coach
- Nombre maximum de participants
- Prix du cours

Lorsqu'un cours est créé, le nombre de participants inscrits est automatiquement à 0.

Exemple :

None

Nom du cours : Yoga débutant

Coach : Emma

Capacité : 12

Prix : 30,50 €

Cours ajouté avec succès.

2. Afficher les cours

Le programme doit afficher tous les cours enregistrés.

Pour chaque cours, afficher :

- Identifiant
- Nom du cours
- Coach
- Places occupées
- Places restantes
- Prix du cours
- Chiffre d'affaire en cours (basé sur les places occupées) et potentiel (basé sur le total des places)

Exemple :

None

#1 - Yoga débutant

Coach : Emma

Places : 3 / 12

Places restantes : 9

Prix du cours : 30,50 €

CA potentiel : 366,00 €

CA actuel : 91,50 €

Si aucun cours n'existe, afficher un message clair.

3. Inscrire une personne à un cours

L'utilisateur doit pouvoir inscrire une personne à un cours.

Le programme doit demander :

None

Identifiant du cours

Si le cours existe et qu'il reste une place :

None

Inscription validée.

Le nombre de participants inscrits augmente de 1.

Si le cours est complet :

None

Impossible : ce cours est complet.

Si le cours n'existe pas :

None

Cours introuvable.

4. Afficher les cours complets

Le programme doit afficher uniquement les cours dont le nombre de participants inscrits est égal à la capacité maximale.

Exemple :

None

Cours complets :

#2 - Pilates

Coach : Nora

Places : 10 / 10

Si aucun cours n'est complet, afficher un message clair.

5. Quitter l'application

L'utilisateur doit pouvoir quitter proprement l'application depuis le menu.

Menu attendu

None

=== Studio Booking Manager ===

1. Créer un cours

2. Afficher les cours

3. Inscrire une personne à un cours

4. Afficher les cours complets

5. Quitter

Votre choix :

Le menu doit continuer à s'afficher tant que l'utilisateur ne choisit pas de quitter.

Contraintes techniques

Votre projet doit s'exécuter depuis un terminal et doit obligatoirement contenir :

Élément	Attendu
POO	au moins une classe représentant un cours
ORM	utilisation d'un ORM pour stocker les cours

Élément	Attendu
Base de données	base SQLite locale
Modules	code séparé dans plusieurs fichiers Python
Boucle	menu interactif qui continue jusqu'à la sortie
Conditions	gestion des choix utilisateur et des cas d'erreur
CRUD partiel	création, lecture et modification de données

La suppression n'est pas demandée.

ORM attendu

L'application doit montrer au minimum :

- Création de la base et de la table
- Ajout d'un cours
- Lecture des cours
- Recherche par identifiant
- Modification du nombre d'inscrits

Organisation attendue du projet

Le projet doit être organisé en plusieurs fichiers. Structurez d'une façon qui vous semble cohérente avec votre code et votre architecture.

Gestion des erreurs attendue

Le programme doit gérer proprement les cas suivants :

- Choix de menu invalide
- Capacité invalide
- Identifiant inexistant
- Cours complet
- Aucun cours enregistré
- Aucun cours complet

Exemple de scénario complet

None

```
=== Studio Booking Manager ===
```

1. Créer un cours
2. Afficher les cours

- 3. Inscrire une personne à un cours
- 4. Afficher les cours complets
- 5. Quitter

Votre choix : 1

Nom du cours : Yoga débutant
Coach : Emma
Capacité : 2
Prix : 30,50 €

Cours ajouté avec succès.

Votre choix : 3

Identifiant du cours : 1

Inscription validée.

Votre choix : 2

#1 - Yoga débutant
Coach : Emma
Places : 1 / 2
Places restantes : 1
Prix du cours : 30,50 €
CA potentiel : 61,00 €
CA actuel : 30,50 €

Votre choix : 3

Identifiant du cours : 1

Inscription validée.

Votre choix : 4

Cours complets :

#1 - Yoga débutant

Coach : Emma

Places : 2 / 2

Votre choix : 5

Fermeture de l'application.

Rendu

Vous devez rendre votre programme sous la forme d'un repository Github (public ou partagé avec moi). Le lien sera à déposer dans le Google Classroom.

Attention, veuillez à bien inclure à votre repository les éléments qui me permettront de recréer votre environnement de développement, sans toutefois inclure les éléments qui sont généralement ignorés dans un projet réel.

Soutenance

Le travail sera évalué sous forme d'une soutenance.

Vous aurez **10 à 15 minutes**, installation comprise, pour présenter votre projet. Il n'est pas nécessaire de préparer des slides : l'objectif est simplement que vous expliquiez ce que vous avez réalisé, comment votre application fonctionne et les grandes parties de votre code.

La présentation durera environ 5 à 7 minutes, puis nous prendrons 5 à 7 minutes pour échanger avec quelques questions.

Le but n'est pas de vous piéger, mais de vérifier que vous comprenez votre travail et les notions vues pendant le module.

Critères d'évaluation

L'évaluation sera notée sur 20 points répartis comme suit :

Programme / Code source (8 points)

Critère	Note
Fonctionnement général Programme exécutable sans erreur bloquante, menu fonctionnel permettant d'utiliser les différentes actions demandées	/2
Programmation orientée objet	/2

Critère	Note
Création pertinente des classes, utilisation correcte des principes de la POO	
Utilisation de l'ORM Création correcte d'un modèle ORM et de la table, utilisation correcte de l'ORM pour lire, créer et modifier les données.	/2
Logique métier Gestion correcte des inscriptions, des cours complets et des cas d'erreur	/1,5
Organisation du projet Respect des consignes sur la modularité du code, utilisation correcte des principes vu en cours pour la gestion des environnements.	/0,5

Soutenance (12 points)

Critère	Note
Présentation et démo Présentation claire du besoin client et du fonctionnement général de l'app. Explication claire de la logique globale suivie. Démonstration fonctionnelle et capacité à expliquer les cas d'erreur et limites du programme.	/4
Compréhension technique Explication correcte de la classe utilisée et de son rôle. Explication correcte du modèle ORM et du lien avec la base de données. Justification des choix d'organisation en modules. Réponses correctes aux questions techniques	/5
Explication de code Capacité à expliquer précisément une partie du code, comment les données sont créées, récupérées et modifiées.	/2
Communication Expression claire, vocabulaire adapté et gestion correcte du temps.	/1